Universidade do Minho
Departamento de Informática

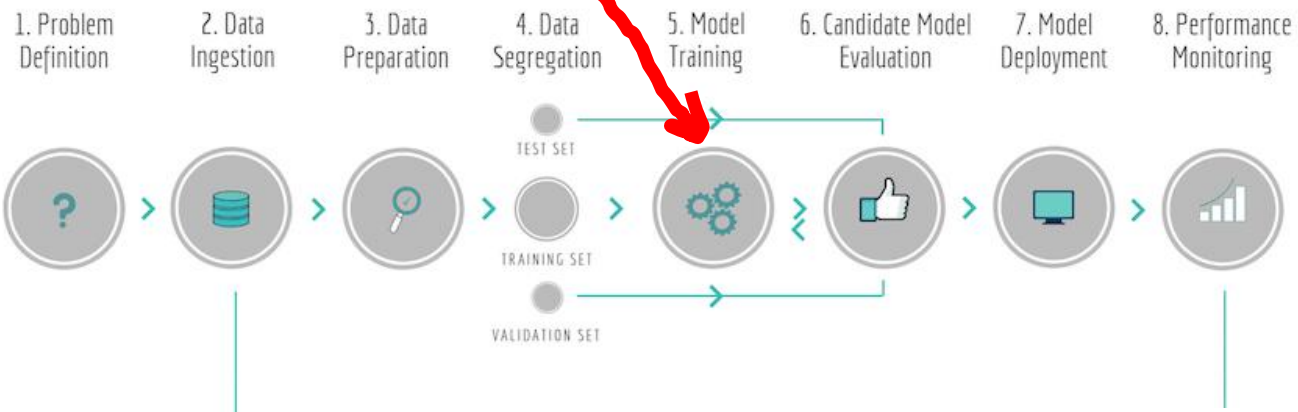
ISLab
Synthetic Intelligence Lab

DADOS e APRENDIZAGEM AUTOMÁTICA

Supervised Learning
Linear and Logistic Regression

Mestrado em Engenharia Informática
Mestrado em Inteligência Artificial
Mestrado em Engenharia e Ciência de Dados
Mestrado em Engenharia de Sistemas
Mestrado em Matemática e Computação
Mestrado em Cibersegurança
Mestrado em Computação Avançada
Mestrado em Redes de Computadores

Contents



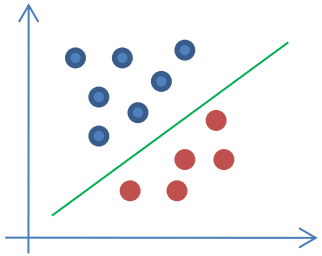
Supervised Learning:

- **Linear Regression;**
- **Logistic Regression**

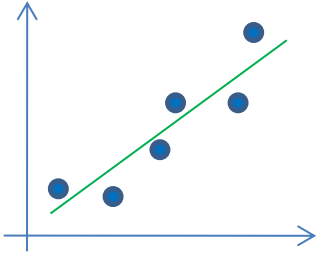
Linear models

- Characterized by the simplicity of calculation and analysis
- Linearity is defined in terms of functions with the properties:

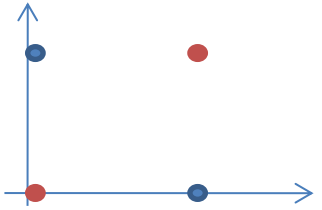
  $f(x + y) = f(x) + f(y)$ and $f(ax) = af(x)$
- Used for **classification** (separation between classes) or **regression**
- Does not solve non-linear problems



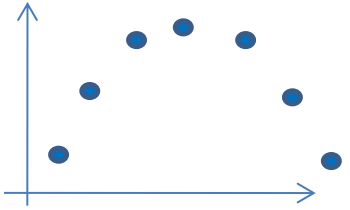
Classification



Regression



Classification
(e.g. XOR)



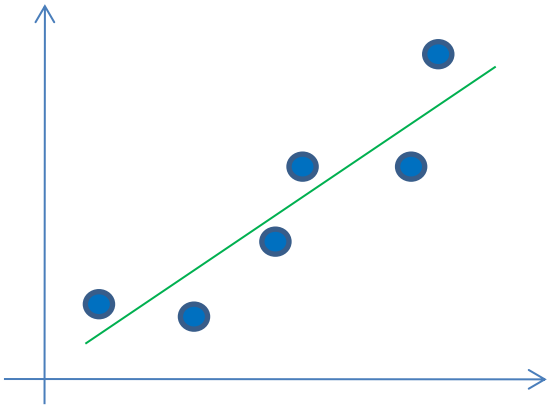
Regression

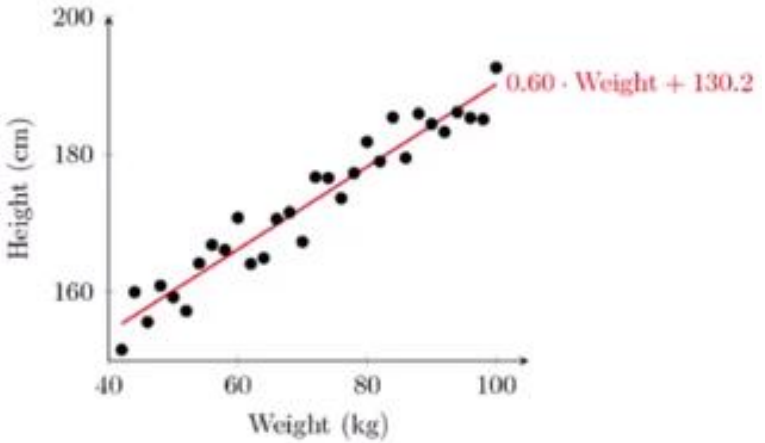
Non-linear problems

Linear Regression

Aims to predict the value of an outcome, Y, based on the value of a predictor variable, X.

- Fit a straight line into a data set of observations;
- Use this line to predict unobserved values.





Linear regression models

Represent the relationship between **input variables** $x_1, \ldots, x_n$ (independent variables), and an **output variable y** (dependent variable).

Model (h) prediction given by (for the **i**-th example):

n – no. of attributes
θ – model parameters

$$\hat{y}^{(i)} = h_\theta(x_1^{(i)}, \ldots, x_n^{(i)})$$

General case: **regression** models

$$\hat{y}^{(i)} = h_\theta(x^{(i)}) = \theta_0 + \sum_{j=1}^{n} \theta_j x_j^{(i)}$$

If **n=1**: **linear regression**

If **n>=2**: **multiple linear regression**

θ_i – model parameters

Multiple linear regression

- Multiple regression is used to determine the effect of a number of independent variables, x_1, x_2, x_3 etc, on a single dependent variable, y
- The different x variables are combined in a linear way, and each has its own regression coefficient (**θ**):

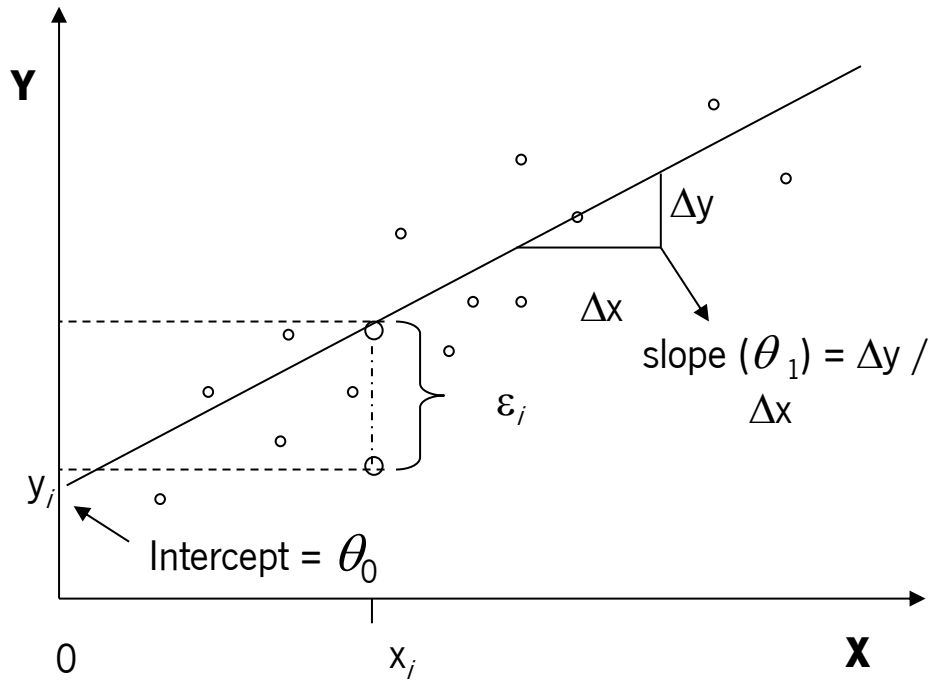
$$\hat{y}^{(i)} = \theta_0 + \sum_{j=1}^{n} \theta_j x_j^{(i)}$$

Model prediction given by (for the **i**-th example)

n – no. of attributes
θ – model parameters

- The **θ** parameters reflect the independent contribution of each independent variable, x, to the value of the dependent variable, y.

Linear Regression



Model (h):

$$\hat{y}^{(i)} = h_\theta(x^{(i)}) = \theta_0 + \sum_{j=1}^{n} \theta_j x_j^{(i)}$$

Universidade do Minho
Departamento de Informática
ISLab
Synthetic Intelligence Lab

Linear Regression Models

How does it work?

Usually using "Error/loss (cost) function" and minimizing its value (minimize the squared-error between each point and the line)

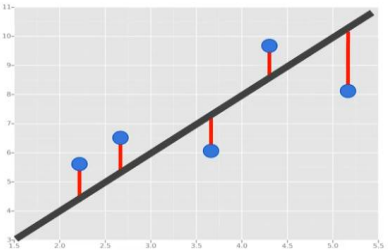
Error/loss (cost) function: mean squares errores (MSE)

$$J_\theta = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

J is a function of the model parameters $\theta_1, \ldots, \theta_n$

$\mathbf{h_\theta(x^{(i)})}$ is the value predicted by the model, $\hat{\mathbf{y}}^{(\mathbf{i})}$

$\mathbf{y^{(i)}}$ is the real value



Objective: to identify the parameters of the model in order to **minimize** the value of **J**

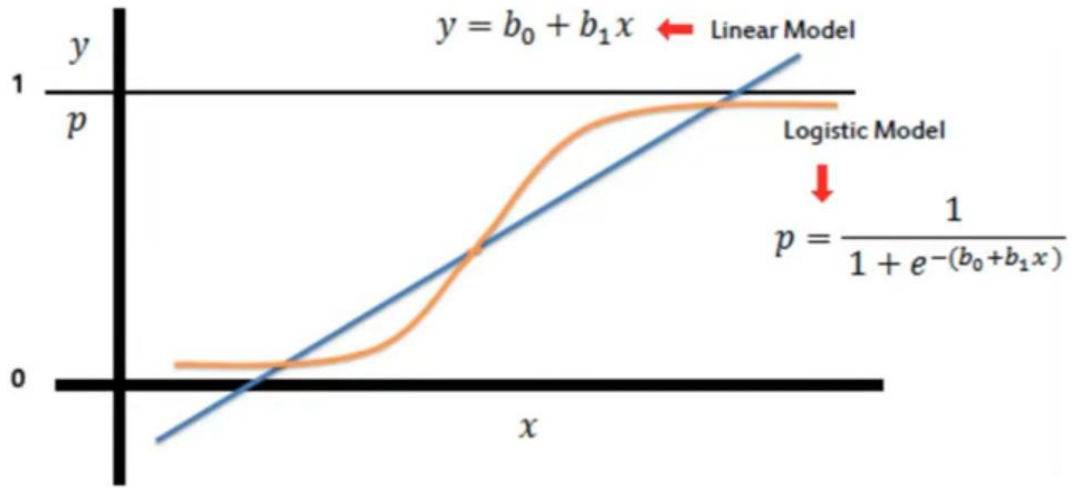
```
#calcula o valor da função de custo (para todos os casos do dataset)
def costFunction(self):
   predictions = np.dot(self.X, self.theta) #produto escalar dos atributos pelos parametros
   J = error_sqe(self.predictions(),self.y) #vai buscar a média dos erros de cada caso
   return J
```

```
def error_sqe(predictions,y):
    m = predictions.shape[0]
    sqe = (predictions - y) ** 2
    res = np.sum(sqe) / (2*m)
    return res
```

Logistic Regression

Discrete dependent variable: **classification** problem

Logistic regression: uses regression models for **binary classification** by interpreting the **model output** in order to extract a **class**



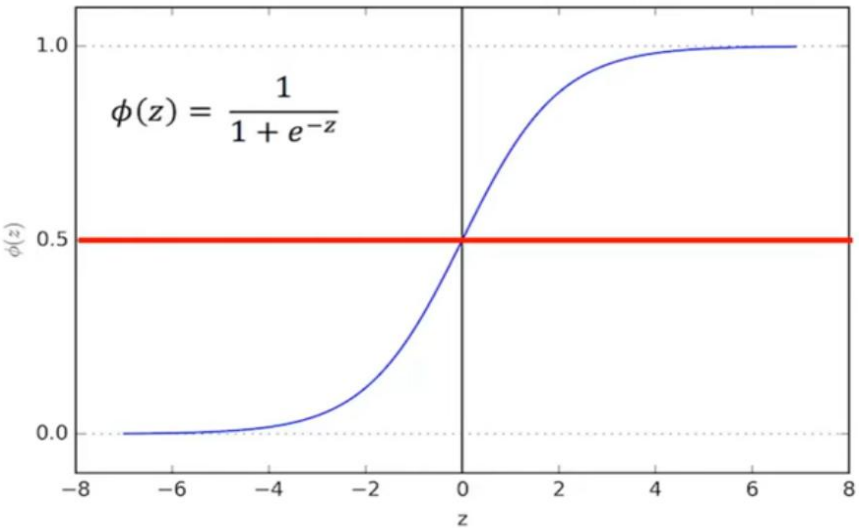
where $\frac{1}{1+e^{-\theta^T x}}$ is the sigmoid (logistic) function

The Model is given by applying the sigmoid to the linear regression function

Interpretation: *p* estimates the probability that **y** (output) **is equal to 1** for example **x**

Sigmoid Function

- The Sigmoid (i.e. Logistic) function takes in any value and outputs it between [0-1];
- This results in a probability from [0-1] of belonging into a class.
- We can set a threshold point at 0.5, defining:
  - ➢ Based off this probability, we assign a class
  - ➢ Predicted results below this threshold results into a class: 0
  - ➢ Predicted results above result results into a class: 1



Logistic Regression: Multiple Classes

- Logistic regression can be applied to cases with **more than two classes**
- In this case, the strategy is **to train a "binary" model for each class separately** (considering the others as a single class)
- Each model estimates the **probability** that **the example is of a given class**
- When predicting new examples, each model is applied by **choosing the class** whose **value predicted** by the model is **greater**.

Logistic Regression: Error Function

Error function (for each example x):

$$\begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

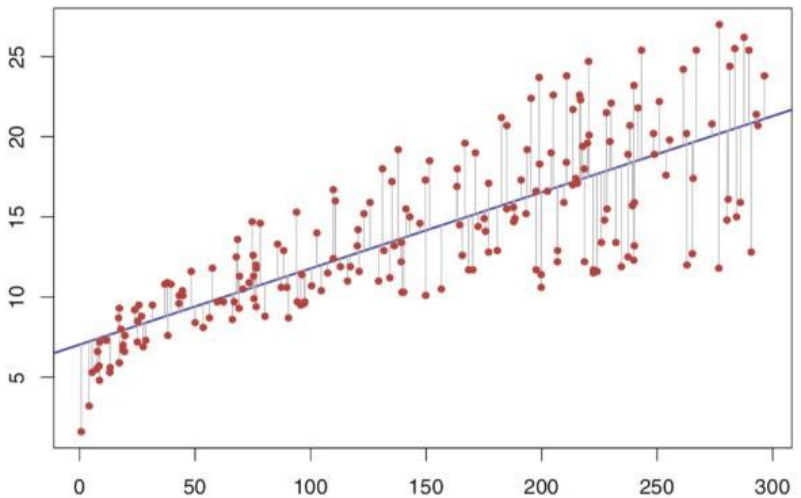
If **y = 1**:
If the prediction is **correct**: **error is zero**
Otherwise, as the prediction gets closer to 0, error **tends to infinity**.

If **y = 0**:
If prediction is correct: **error is zero**
Otherwise, as the prediction gets closer to 1, error **tends to infinity**.

Parameter Estimation: Optimization

Knowing the model structure: parameter estimation is a **numerical optimization problem** – error function minimization

In the case of linear models, the **least squares method** can be used, which minimizes the error function (square of errors) or iterative method



Parameter Estimation: Least Squares Method for Linear Regression

Analytical method to determine optimal values of parameters that minimize **J**

Algebraic method that involves solving a system of equations given by:

$$\frac{\partial}{\partial \theta_j} J(\theta) = 0, j = 1,\ldots,n \qquad \theta = (X^T X)^{-1} X^T y$$

Matricial version; **X** matrix includes examples **+ 1st collumn of 1's**

The **computational complexity** when training a linear regression model using the least squares method is **linear** with respect to the number of instances and features.

```
def buildModel(self):  #cria o modelo usando o método analítico
   from numpy.linalg import inv #biblioteca Linear Algebra do numpy
                                #inv - Compute the (multiplicative) inverse of a matrix.
                                #dá uma matriz que ao multiplicar pela matriz retorna a matriz identidade
   self.theta = inv(self.X.T.dot(self.X)).dot(self.X.T).dot(self.y)
```

Universidade do Minho
Departamento de Informática
ISLab
Synthetic Intelligence Lab

Parameter Estimation: Gradient Descent for Linear Regression

Method that depends on whether the error function is differentiable

Iterative method, which in each iteration changes the values of each of the parameters θ_j

For each θ_j the update rule is as follows:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Simultaneous updates on all parameters

J is the error, h is the model and x is the vector with the attributes, m is the number of instances

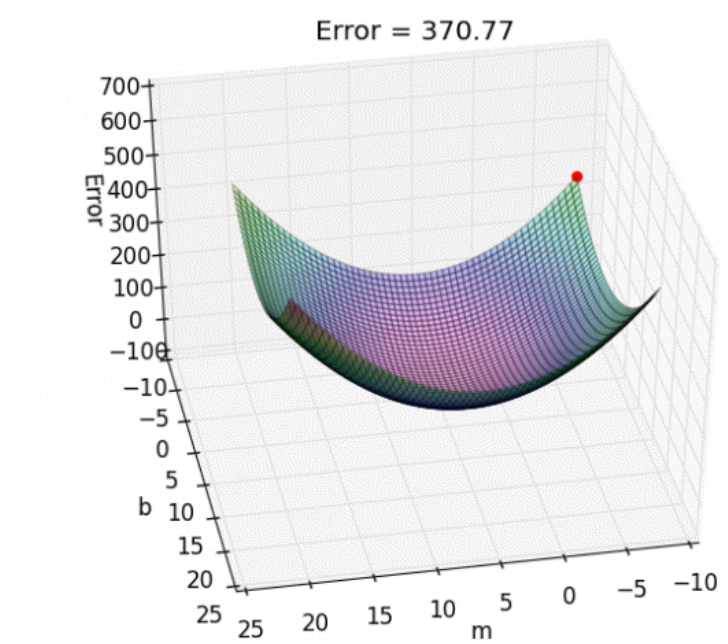
The method starts the theta (parameters) with random values and improves gradually at each iteration, taking a small step at each iteration. The size of the step is called "learning rate (α)".

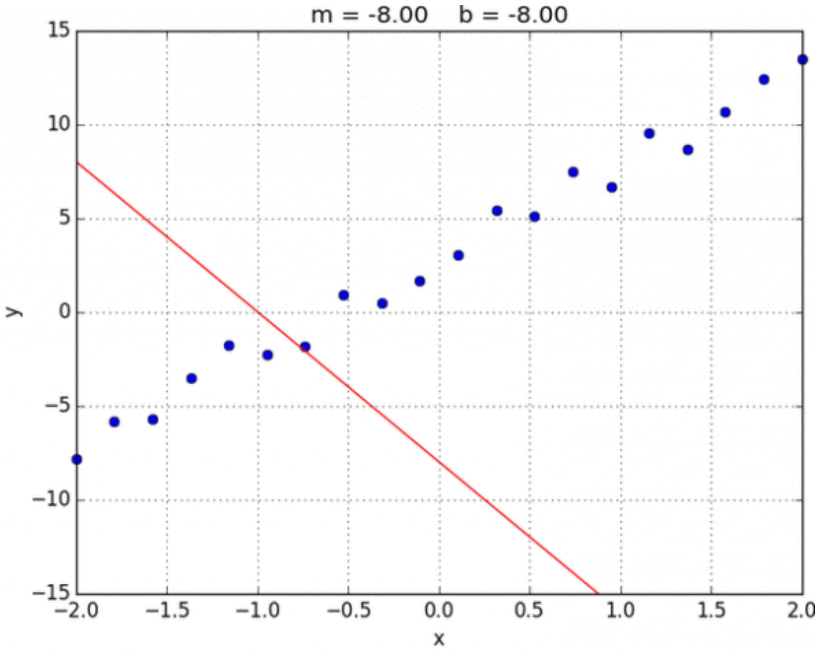
To implement the gradient descent, we need to calculate how much the cost function will change if we change just a little bit of the parameters (theta). This is called the partial derivative. The derivative gives us the rate of change of a function at a given point. When we have a variation equal to zero it means we have reached the objective of the function.

```
def gradientDescent (self, iterations = 1000, alpha = 0.001): #cria o modelo usando gradiente - descendente
    m = self.X.shape[0] #numero de casos
    n = self.X.shape[1] #numero de atributos
    self.theta = np.zeros(n) #os parametros inicializam a 0
    for its in range(iterations):
        J = error_sqe(self.predictions(),self.y) #vai buscar a média dos erros de cada caso
        if its%100 == 0: print(f"Erro na iteração {its} : {J}")
        delta = self.X.T.dot(self.X.dot(self.theta) - self.y)
        self.theta -= (alpha/m * delta )
```

Universidade do Minho
Departamento de Informática
ISLab
Synthetic Intelligence Lab

Parameter Estimation: Gradient Descent for Linear Regression



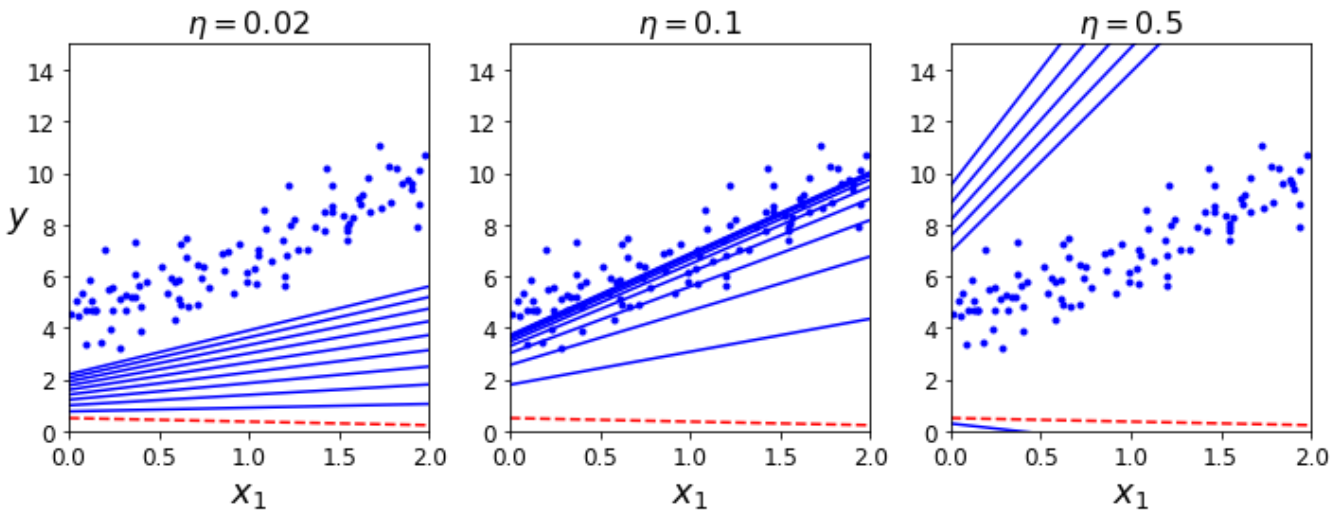


Parameter Estimation: Gradient Descent

The parameter **α** is called the **learning rate** and controls the **"speed"** of updating the parameters

Lower α values guarantee convergence but it may be slower

Higher α values can lead to **faster convergence**, but carry risks of divergence



Parameter Estimation: Gradient Descent vs. Analytic Method

Analytical method guarantees the optimal solution; **GD may not converge**

In the analytical method there are **no parameters**; **GD may take time to converge**

Analytical method can become slow with **very large n** (**n×n matrices** can become **intractable** for **n>105**)

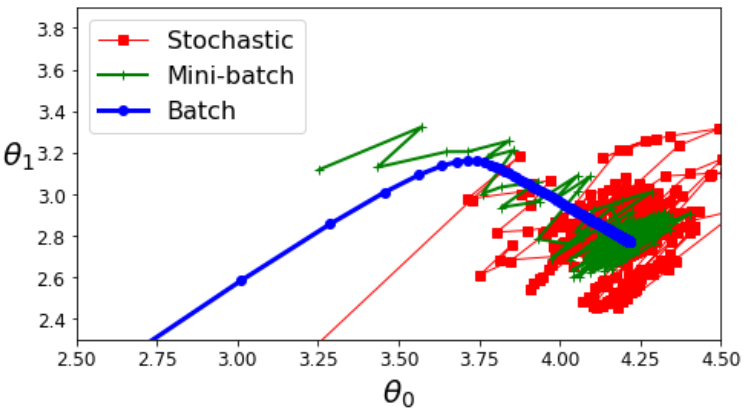
More generic GD are applicable to other types of models

Parameter Estimation: Advanced Methods

In many cases, gradient descent is **too slow to converge** to be used in practice

Other more advanced numerical optimization methods can be used

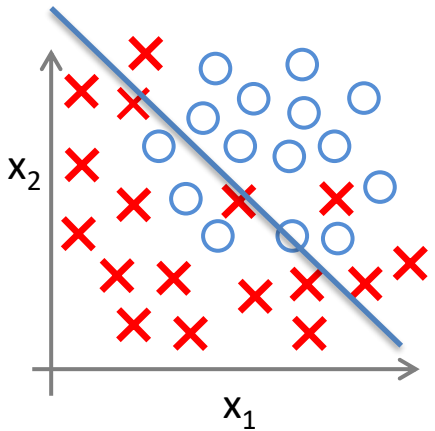
Python example with the **fmin** function from package **optimize** (in this case, no derivatives needed). Other alternatives available in the same package



Algorithm	Many Instances (m)	Many Attributes (n)
Square Min.	Fast	Slow
Batch GD (all data at each iteration)	Lento	Fast
Stocastic GD (one instance at each iteration)	Fast	Fast
Mini Batch GD (random data at each iteration)	Fast	Fast

Universidade do Minho
Departamento de Informática
ISLab
Synthetic Intelligence Lab

Overfitting in Logistic Regression: example

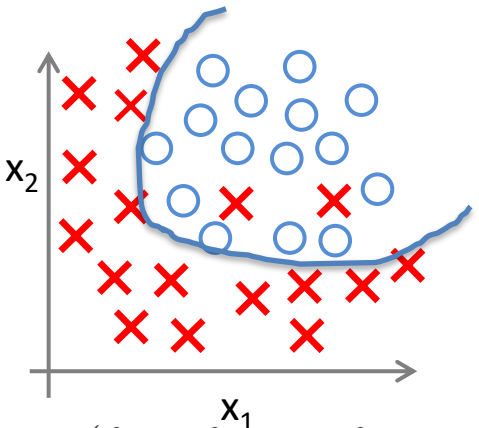


$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

g is a sigmoid function

Underfitting:
insuficient complexity

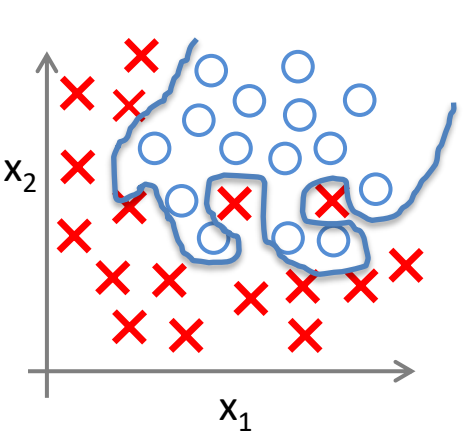
Linear Regression



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$$

"Adequate" complexity

Polynomial Regression



$$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$$

Overfitting:
excessive complexity

Linear Regression with Interaction Terms

Universidade do Minho
Departamento de Informática
ISLab
Synthetic Intelligence Lab

Solutions for Overfitting: Functional Models

Reduce the number of attributes (coefficients) used
Select attributes "**manually**" by knowledge of the problem
Attribute selection algorithms

Regularization:
Keep all attributes but **try to reduce magnitude** of parameter value

Linear Regression $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$

The objective is to minimize the mean squared error:

$$\min_{\beta} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Does not apply any form of regularization and may suffer from overfitting if there are many irrelevant variables or high collinearity.

Linear Regression: fits only a line/plane without regularization.

Lasso Regression (L1): It is a variation of linear regression that adds **L1 regularization** to the optimization problem, which helps with automatic feature selection and model complexity control. Forces some coefficients exactly to zero →performs automatic feature selection.

$$\min_{\beta} \left[\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right]$$

- $\lambda$is the regularization parameter.
- The term $\sum |\beta_j|$ is the L1 penalty.

Ridge Regression (L2): shrinks the coefficients, but does not set them to zero →keeps all variables in the model, but reduces the impact of the less relevant ones.

In practical terms: if we have a dataset with many features, and not all of them are relevant, Lasso tends to automatically "choose" the most important ones.

Standardization and Normalization

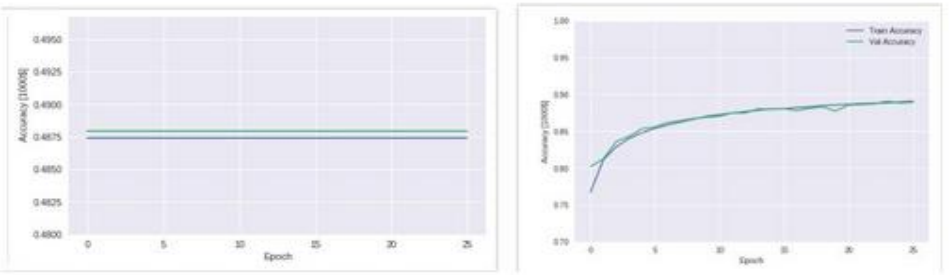
Transformations in the data are often necessary for the learning algorithm to **work better**

Gradient descent algorithms may **work worse** with variables with v**ery different scales**

Several possible methods:

- Convert to **mean 0** and **standard deviation 1**
- Convert to a **[0,1]** or **[-1,1] range**, setting **minimum and maximum values**

Left: Model Accuracy, without normalized data
Right: Model Accuracy with normalized data



Universidade do Minho
Departamento de Informática

ISLab
Synthetic Intelligence Lab

DADOS e APRENDIZAGEM AUTOMÁTICA

Supervised Learning
Linear and Logistic Regression

Mestrado em Engenharia Informática
Mestrado em Inteligência Artificial
Mestrado em Engenharia e Ciência de Dados
Mestrado em Engenharia de Sistemas
Mestrado em Matemática e Computação
Mestrado em Cibersegurança
Mestrado em Computação Avançada
Mestrado em Redes de Computadores